

P6: WAP to implement fractional knapsack problem using greedy approach

```
import java.util.Arrays;
```

```
import java.util.*;
```

```
class Item {
```

```
    int value, weight;
```

```
    Item(int value, int weight) {
```

```
        this.value = value;
```

```
        this.weight = weight;
```

```
    }
```

```
}
```

```
public class P6_Fractional_knapsack {
```

```
    public static double knapsackMinWeight(Item[] items, int capacity) {
```

```
        Arrays.sort(items, (a, b) -> a.weight - b.weight);
```

```
        return fillKnapsack(items, capacity);
```

```
    }
```

```
    public static double knapsackMaxProfit(Item[] items, int capacity) {
```

```
        Arrays.sort(items, (a, b) -> b.value - a.value);
```

```
        return fillKnapsack(items, capacity);
```

```
    }
```

```
    public static double knapsackRatio(Item[] items, int capacity) {
```

```
        Arrays.sort(items, (a, b) -> Double.compare((double) b.value / b.weight, (double) a.value / a.weight));
```

```
        return fillKnapsack(items, capacity);
```

```
    }
```

```
    private static double fillKnapsack(Item[] items, int capacity) {
```

```

double totalValue = 0.0;
for (Item item : items) {
    if (capacity == 0)
        break; // Knapsack full
    if (item.weight <= capacity) {
        totalValue += item.value;
        capacity -= item.weight;
    } else {
        totalValue += (double) item.value * capacity / item.weight;
        break;
    }
}
return totalValue;
}

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of items: ");
    int n = sc.nextInt();

    Item[] items = new Item[n];
    for (int i = 0; i < n; i++) {
        System.out.print("Enter value and weight of item " + (i + 1) + ": ");
        int value = sc.nextInt();
        int weight = sc.nextInt();
        items[i] = new Item(value, weight);
    }

    System.out.print("Enter knapsack capacity: ");
    int capacity = sc.nextInt();
}

```

```

        System.out.println("\n--- Fractional Knapsack Results ---");

        System.out.println("Case 1 (Min Weight) = " + knapsackMinWeight(items.clone(), capacity));

        System.out.println("Case 2 (Max Profit) = " + knapsackMaxProfit(items.clone(), capacity));

        System.out.println("Case 3 (Profit/Weight) = " + knapsackRatio(items.clone(), capacity));

        sc.close();
    }
}

```

Output:

```

Enter number of items: 3
Enter value and weight of item 1: 60
10
Enter value and weight of item 2: 100
20
Enter value and weight of item 3: 120
30
Enter knapsack capacity: 50

--- Fractional Knapsack Results ---
Case 1 (Min Weight) = 240.0
Case 2 (Max Profit) = 220.0
Case 3 (Profit/Weight) = 240.0

```

P7: Write a program to implement 0/1 Knapsack Problem using Dynamic Programming.

```

import java.util.Scanner;

public class P7_01Knapsack {

    public static int knapsack(int W, int[] wt, int[] val, int n) {

        int[][] dp = new int[n + 1][W + 1];

        for (int i = 0; i <= n; i++) {

            for (int w = 0; w <= W; w++) {

                if (i == 0 || w == 0)

```

```

        dp[i][w] = 0;

    else if (wt[i - 1] <= w)

        dp[i][w] = Math.max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);

    else

        dp[i][w] = dp[i - 1][w];

    }

}

int res = dp[n][W];

int w = W;

System.out.print("Items chosen (value, weight): ");

for (int i = n; i > 0 && res > 0; i--) {

    if (res == dp[i - 1][w])

        continue; // item not included

    else {

        System.out.print("(" + val[i - 1] + ", " + wt[i - 1] + ") ");

        res -= val[i - 1];

        w -= wt[i - 1];

    }

}

System.out.println();

return dp[n][W];

}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of items: ");

    int n = sc.nextInt();

    int[] val = new int[n];

    int[] wt = new int[n];

```

```

for (int i = 0; i < n; i++) {

    System.out.print("Enter value and weight of item " + (i + 1) + ": ");

    val[i] = sc.nextInt();

    wt[i] = sc.nextInt();

}

System.out.print("Enter Knapsack Capacity: ");

int W = sc.nextInt();

int maxProfit = knapsack(W, wt, val, n);

System.out.println("Maximum profit in 0/1 Knapsack = " + maxProfit);

sc.close();

}

}

```

Optput:

```

Enter number of items: 3
Enter value and weight of item 1: 60
10
Enter value and weight of item 2: 100
20
Enter value and weight of item 3: 120
30
Enter Knapsack Capacity: 50
Items chosen (value, weight): (120,30) (100,20)
Maximum profit in 0/1 Knapsack = 220

```